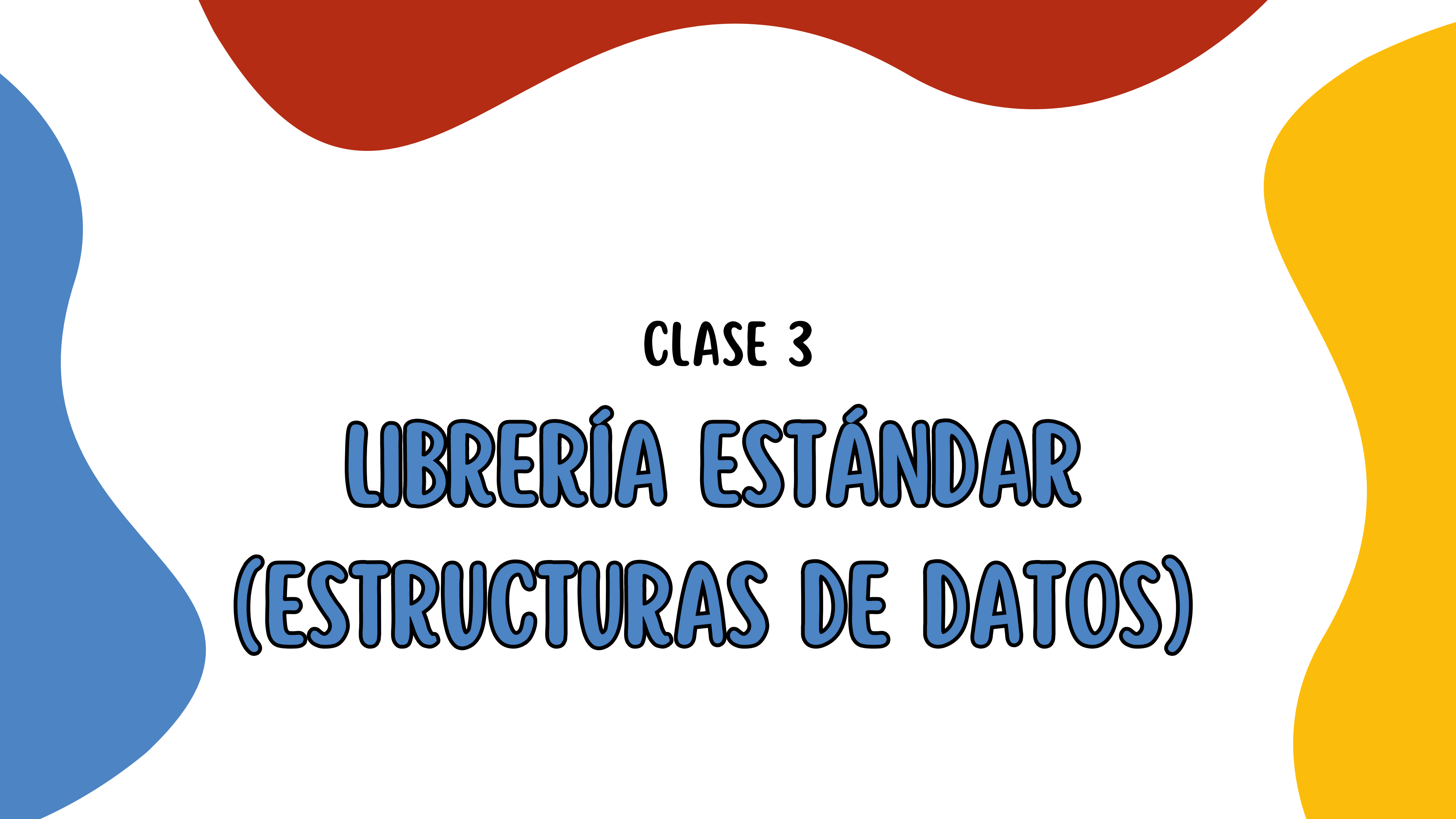




CLASE 3

LIBRERÍA ESTÁNDAR
(ESTRUCTURAS DE DATOS)

ESTRUCTURAS DE DATOS

Una estructura de datos es una forma de organizar información, de manera que podamos acceder a ella convenientemente.

CUÁL USAR

La mejor estructura de datos va a depender de las necesidades del problema. A veces puede ser difícil saber cuál escoger, pero la práctica hace al maestro.

DEFINICIÓN FORMAL

Una estructura de datos es un objeto que nos permite realizar operaciones básicas.

Por ejemplo, la lista nos permite:

- Agregar un elemento al final de la lista.
- Quitar el elemento final de la lista.
- Leer y escribir elementos usando su índice.

COMPLEJIDAD

Cada operación tiene asociada una complejidad, es decir, algún tipo de garantía del tiempo que tomará realizarla.

Por ejemplo, la lista:

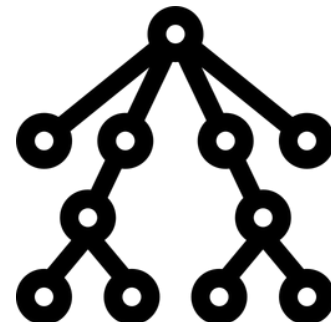
- Agregar un elemento al final de la lista: $O(1)$ ₊
- Quitar el elemento final de la lista: $O(1)$ ₊
- Leer y escribir elementos usando su índice: $O(1)$

DETRÁS DE ESCENAS

Las estructuras de datos son librerías, funciones y código como cualquier otro, y alguien tiene que implementarlas.



TÚ



ESTRUCTURA
DE DATOS



101
011

IMPLEMENTACIÓN

LISTAS

Un arreglo dinámico. Los elementos están almacenados de forma contigua en memoria.

- **Agregar un elemento al final de la lista: $O(1)$**
- **Quitar el elemento final de la lista: $O(1)$**
- **Leer y escribir elementos usando su índice: $O(1)$**
- **Ordenar los elementos: $O(n \log n)$**

LISTAS: PROBLEMA

Tienes n diferentes tipos de dulces en tu tienda. Vas a recibir clientes, los cuales te van a decir cuales dulces quieren, y cuántos de cada uno. Debes calcular cuánto cobrar a cada cliente

MAPA

Esta estructura asocia una llave a un valor. Básicamente, es un arreglo, pero en vez de indexar con enteros desde el 0, indexamos con cualquier cosa.

- **Agregar un elemento : $O(\log n)$**
- **Quitar un elemento: $O(\log n)$**
- **Leer y escribir elementos: $O(\log n)$**
- **Consultar si una key tiene algo asociado: $O(\log n)$**
- **Iterar por los elementos ordenados según su key: $O(n)$**

MAPA DE HASH

También asocia una llave a un valor. Sin embargo, son implementaciones distintas. Este se basa en probabilidades (hash): probablemente es rápido aunque a veces puede ser lento.

- Agregar un elemento : $O(1)$?
- Quitar un elemento: $O(1)$?
- Leer y escribir elementos: $O(1)$?
- Consultar si una key tiene algo asociado: $O(1)$?
- **No** itera por los elementos ordenados según su key

MAPA: PROBLEMA

Eres el encargado de llevar la cuenta de la cantidad de productos en el supermercado. Debes tener en cuenta que puedes abastecerte de productos, vender productos y ser capaz de decirle al cliente si tenemos suficientes productos para llevar a cabo su compra.

SET

Conjunto de elementos ordenado. No permite tener duplicados: si agrego un elemento varias veces, entonces solo aparece una vez.

- **Agregar un elemento : $O(\log n)$**
- **Quitar un elemento: $O(\log n)$**
- **Consultar si un elemento dado está en el set: $O(\log n)$**
- **Buscar el primer elemento $\geq$ a un valor dado: $O(\log n)$**
- **Iterar por los elementos ordenados según su key: $O(n)$**

MULTISET

Conjunto de elementos ordenado. **Permite tener duplicados.**

- **Agregar un elemento : $O(\log n)$**
- **Quitar un elemento: $O(\log n)$**
- **Consultar si un elemento dado está en el set: $O(\log n)$**
- **Buscar el primer elemento $\geq$ a un valor dado: $O(\log n)$**
- **Iterar por los elementos ordenados según su key: $O(n)$**

SET DE HASH

Conjunto de elementos ~~ordenado~~. No permite tener duplicados: si agrego un elemento varias veces, entonces solo aparece una vez.

- Agregar un elemento : $O(1)?$
- Quitar un elemento: $O(1)?$
- Consultar si un elemento dado está en el set: $O(1)?$

SET: PROBLEMA

Voy a tener la recta numérica en la que voy a ir construyendo casas. Antes de construir una casa, quiero saber si hay ya una casa en esa posición. Además, me gustaría saber dónde estaría mi vecino a la izquierda y a la derecha.

PILA (STACK)

Estructura de datos LIFO (Last In, First Out). Imagina una pila de platos: solo puedes añadir (push) o quitar (pop) un plato de la parte superior.

- Agregar un elemento a la parte superior: $O(1)$
- Consultar por el elemento de la parte superior: $O(1)$
- Quitar el elemento de la parte superior: $O(1)$

PILA: PROBLEMA

Dada una cadena que contiene solo los caracteres (,), {, }, [y], determina si la cadena de entrada es válida.

Una cadena es válida si:

- 1. Los paréntesis abiertos deben cerrarse con el mismo tipo.**
- 2. Los paréntesis abiertos deben cerrarse en el orden correcto.**

COLA (QUEUE)

Estructura de datos FIFO (First In, First Out). Como una fila en el cine: la primera persona en llegar es la primera en ser atendida.

- **Agregar un elemento al final de la cola: $O(1)$**
- **Consultar por el elemento del frente: $O(1)$**
- **Sacar el elemento del frente: $O(1)$**

COLA: PROBLEMA

Se te da una cuadrícula donde cada celda puede tener una naranja fresca (1), una naranja podrida (2) o estar vacía (0). Cada minuto, cualquier naranja fresca que sea adyacente (4 direcciones) a una naranja podrida se pudre. Responde cual es la ultima naranja en podrirse (si hay mas de una que se pudrieron al mismo tiempo al final, cualquiera de esas sirve).

COLA DE PRIORIDAD

Una cola especial donde los elementos con mayor prioridad se atienden antes que los de menor prioridad, independientemente del orden de llegada.

- Agregar un elemento: $O(\log n)$
- Consultar por el elemento de mayor prioridad: $O(1)$
- Quitar el elemento de mayor prioridad: $O(\log n)$

COLA DE PRIORIDAD: PROBLEMA

Hay n comidas. Cada comida tiene un precio p , y una cantidad q de comidas que debieron ser comidas antes de esta (solo la cantidad, no importa cuales). Quieres comer k comidas al costo mas barato posible.